d C++

Debugging with LLDB

Posted on April 13, 2015

We talked about debugging with valgrind last time, and saw how we can catch an insidious kind of memory usage error called a segfault with its `memcheck` tool.

However, some were quick to comment that `valgrind` isn't the best tool for diagnosing segfaults. This is true – using `valgrind` to figure out what's causing a segfault is somewhat like using a giant fishnet to catch a single large-mouth bass. What you really want is a nice, flexible fishing rod, some powerful fishing line and some especially tasty worms. `valgrind` is actually doing much more than just looking for invalid memory accesses; it's also tracking memory use to record leaks, use of uninitialized memory, and more – all very expensive operations, and operations that might not be of primary interest when you're just tracking down a crash.

There are two more 'targetted' debuggers, `gdb` and `lldb`, which become much more useful when you need to:

1. Walk through C / C++ code execution one line, or block, at a time,
2. Run a script and have it immediately 'break' (stop execution) when the segfault is encountered, thereby allowing you to inspect the current

state of variables, the stack, and otherwise – giving you the ability to **interactively** investigate what's going on at the time of the segfault.

This post will be a brief introduction to `lldb` as used on OS X; what you learn here will (mostly) be applicable to `gdb` on UNIX-alikes as well – they perform essentially the same function, although `lldb` is somewhat behind `gdb` in terms of advanced features.

If you're running on Linux, you can either try installing `lldb` yourself (`sudo apt-get install lldb`), or just using `gdb` – `lldb` intentionally copies `gdb` 's interface for many common commands, although `lldb` chooses to be verbose for most commands (and considers the `gdb` -type interface as 'shortcuts').

Training Wheels

First, let's try starting R with `lldb` 'attached' – that is, with `lldb` monitoring the R process and watching to see if it tries to do anything naughty. It's similar to what we used for `valgrind` – let's try:

```
R -d lldb
```

You should see something like:

```
kevin:~$ /usr/bin/R -d lldb
(lldb) target create "/Library/Frameworks/R.framework/Resources/bin/e
Current executable set to '/Library/Frameworks/R.framework/Resources/
(lldb) |
```

You'll notice one difference between the previous post – we aren't immediately bounced into R; instead, we are planted into an `lldb` REPL – with `(lldb)` as the prompt showing that. This allows us to set up how we want `lldb` to handle this current R process; but let's instead just jump into R. We can do this by calling `run`. You should see something like:

```
(lldb) run
Process 3158 launched: '/Library/Frameworks/R.framework/Resources/bir

R Under development (unstable) (2015-04-05 r68149) -- "Unsuffered Cor
Copyright (C) 2015 The R Foundation for Statistical Computing
Platform: x86_64-apple-darwin14.1.0 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

  Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

>
```

And you are now in your regular old R console, and can run things as per usual.

Now, let's start breaking things. We'll use `Rcpp` to cause a segfault, by

writing to an invalid index again. We'll do the same thing we did last time – write out of bounds in a `Rcpp::NumericVector` :

```
Rcpp::cppFunction("
  SEXP ouch() {
    NumericVector x;
    x[1000000] = 1;
    return x;
  }
")
```

If you then call the `ouch()` function, you *should* have `lldb` immediately plop you down right where the failure happened.

(Side note: ironically, trying to 'reproduce' this later, I was unable to get this function to segfault within `lldb` , while `valgrind` still reported an invalid write (even if it didn't segfault). This is why, when it comes to debugging C / C++ bugs, you really need a wide arsenal of tools! Or, you might even need, for example):

```
repeat { ouch() }
```

to **really** get R to throw up.

Back to our main scheduled program – what do we see when `lldb` stops execution?

```
rep> repeat { ouch() }
Process 16065 stopped
* thread #1: tid = 0x19671, 0x0000000108ff6344 sourceCpp_75751.so`ouc
```

```
    frame #0: 0x0000000108ff6344 sourceCpp_75751.so`ouch() + 244 at f
   6
   7          SEXP ouch() {
   8            NumericVector x;
-> 9            x[10000000] = 1;
   10           return x;
   11         }
   12
```

Isn't that nice? The output shows us exactly where the error happened, but instead of the stack trace dumped by `valgrind`, we are now actually plopped back into the `lldb` REPL, and can now examine what each of the various executing threads and frames look like. To get the similar stack trace that we saw from `valgrind`, we can call `bt` or `thread backtrace`. This will show us the 'stack trace', or the layers of executing function calls, that lead us to the segfault:

```
(lldb) bt
* thread #1: tid = 0x19671, 0x0000000108ff6344 sourceCpp_75751.so`ouc
  * frame #0: 0x0000000108ff6344 sourceCpp_75751.so`ouch() + 244 at f
    frame #1: 0x0000000108ff6414 sourceCpp_75751.so`::sourceCpp_64335
    frame #2: 0x000000010007e5e7 libR.dylib`do_dotcall(call=0x000000
    frame #3: 0x00000001000a9f2c libR.dylib`Rf_eval(e=0x000000010839
    frame #4: 0x00000001001008e8 libR.dylib`Rf_applyClosure(call=0x00
    frame #5: 0x00000001000aa067 libR.dylib`Rf_eval(e=0x00000001094c
    frame #6: 0x0000000100103813 libR.dylib`do_begin(call=0x000000010
    frame #7: 0x00000001000aa15e libR.dylib`Rf_eval(e=<unavailable>,
    frame #8: 0x00000001001035ec libR.dylib`do_repeat(call=<unavailab
    frame #9: 0x00000001000aa15e libR.dylib`Rf_eval(e=<unavailable>,
    frame #10: 0x000000010013528f libR.dylib`Rf_ReplIteration(rho=0x0
    frame #11: 0x0000000100136733 libR.dylib`run_Rmainloop [inlined]
    frame #12: 0x00000001001366d2 libR.dylib`run_Rmainloop + 98 at ma
    frame #13: 0x0000000100000f3b R`main(ac=<unavailable>, av=<unavai
    frame #14: 0x00007fff8ad055c9 libdyld.dylib`start + 1
```

```
    frame #15: 0x00007fff8ad055c9 libdyld.dylib`start + 1
```

It's kind of interesting to see how many layers of evaluation there are, until we actually touch the function exported by our `cppFunction()` call. However, just like with `valgrind`, we have information + line numbers on exactly where things went wrong.

That gives us the birds-eye overview of how `lldb` can be used:

1. Use `R -d lldb` to launch R with `lldb` attached,
2. Use `run` to start the R process,
3. Execute some code that triggers the segfault.

Learning to Drive

Now, let's explore what is probably the primary use of `lldb` – setting breakpoints. Sometimes, we want to be able to just stop code execution at arbitrary lines of code, and inspect what's going on. We do this by setting breakpoints either on the file + line number, or the function name itself. I'll focus on the second version.

Let's launch R again, with `lldb` attached:

```
R -d lldb
```

and then type `run` to start the R process.

Now that we're in R, we're going to pop back into `lldb` and try to set a breakpoint. Try pressing `Ctrl + C` – this interrupts the execution of the R

REPL, and puts the `lldb` REPL back in control:

```
> Process 3885 stopped
* thread #1: tid = 0x63cc, 0x00007fff8a7193fa libsystem_kernel.dylib`
    frame #0: 0x00007fff8a7193fa libsystem_kernel.dylib`__select + 1(
libsystem_kernel.dylib`__select:
->  0x7fff8a7193fa <+10>: jae    0x7fff8a719404           ; <+20>
    0x7fff8a7193fc <+12>: movq   %rax, %rdi
    0x7fff8a7193ff <+15>: jmp    0x7fff8a714c78           ; cerror
    0x7fff8a719404 <+20>: retq
```

What you're seeing there are the plain assembly instructions that are getting executed where we stopped. In this case, we're right in the system kernel, and so we don't have any good debugging information available.

Let's step back into R for a second. Type `continue`, or `c`, and press ENTER and you should see the R console come back to life (and press ENTER again to get the prompt to reappear):

```
(lldb) continue
Process 3885 resuming

>
```

Let's see what `Rcpp` does behind the scenes when it attempts to compute the mean of a vector. Let's compile a simple function calling the Rcpp sugar mean, as:

```
Rcpp::cppFunction("double rcppMean(NumericVector x) { return mean(x);
```

Pop back into `lldb` with `Ctrl + C`, and now we will set a breakpoint on our `rcppMean` function. You can write `breakpoint set --name rcppMean` if you enjoy being verbose, or the more simple `b rcppMean` to do the same with fewer characters. Also, it's worth pointing out that `lldb` will also do `TAB` completion for you; that is, you should see that `b rcppM<TAB>` provides `rcppMean` as an autocomplete suggestion. That's nice!

After writing this command, you should see:

```
(lldb) b rcppMean(Rcpp::Vector<14, Rcpp::PreserveStorage>)
Breakpoint 1: 2 locations.
```

which tells that you that `lldb` has found a function by the name of `rcppMean`, and is ready to pause execution if anyone calls it. Now let's ask R to call it!

Step back into R by entering `thread continue` or `c` into the LLDB repl, and try calling `rcppMean(1:5)`:

```
(lldb) c
Process 4169 resuming

> rcppMean(1:5)
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7dad1 sourceCpp_57725.so`::sc
    frame #0: 0x000000010cf7dad1 sourceCpp_57725.so`::sourceCpp_12391
   3    using namespace Rcpp;
   4
   5    // [[Rcpp::export]]
-> 6    double rcppMean(NumericVector x) { return mean(x); }
   7
   8
```

```
    9    #include <Rcpp.h>
```

Okay, cool, that's our function call – but what we really want to see is the guts of `Rcpp` 's implementation of the `mean` function. We can **s**tep into that function by executing `thread step-in` , or `s` for short:

```
(lldb) s
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e644 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e644 sourceCpp_57725.so`Rcpp::sugar::Mear
   34       Mean(const VEC_TYPE& object_) : object(object_) {}
   35
   36       double get() const {
-> 37           VECTOR input = object;
   38           int n = input.size();           // double pass (as ir
   39           long double s = std::accumulate(input.begin(), input.
   40           s /= n;
```

Sweet! We see the code that was being executed and exactly where we are in running through that code. Let's now move to the **n**ext line, with `thread step-over` or `n` :

```
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e75c sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e75c sourceCpp_57725.so`Rcpp::sugar::Mear
   37           VECTOR input = object;
   38           int n = input.size();           // double pass (as ir
   39           long double s = std::accumulate(input.begin(), input.
-> 40           s /= n;
   41           if (R_FINITE((double)s)) {
   42               long double t = 0.0;
```

```
    43                for (int i = 0; i < n; i++) {
```

Hm! In my case, we actually skipped a bunch of instructions – this kind of thing can happen if the compiler has optimized out certain steps. Unfortunately, after an optimizing compiler gets done with your code, the actual instructions generated could have almost nothing to do with your source code, and so it is no longer possible to provide a 1-1 mapping between the actual instructions generated and your original source code. (This is why you may consider producing so-called 'debug' builds, with optimization toned down, when attempting to debug these kinds of issues as well. For building R packages, this effectively amounts to something like `CXXFLAGS=-g -O0` in your `~/.R/Makevars` file)

This is fun! Let's step through a bit more.

```
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e76a sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e76a sourceCpp_57725.so`Rcpp::sugar::Mear
   38           int n = input.size();           // double pass (as ir
   39           long double s = std::accumulate(input.begin(), input.
        s /= n;
-> 41           if (R_FINITE((double)s)) {
   42               long double t = 0.0;
   43               for (int i = 0; i < n; i++) {
   44                   t += input[i] - s;
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e782 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e782 sourceCpp_57725.so`Rcpp::sugar::Mear
   40           s /= n;
   41           if (R_FINITE((double)s)) {
```

```
   42                long double t = 0.0;
-> 43                for (int i = 0; i < n; i++) {
   44                    t += input[i] - s;
   45                }
   46                s += t/n;
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e790 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e790 sourceCpp_57725.so`Rcpp::sugar::Mear
   41            if (R_FINITE((double)s)) {
   42                long double t = 0.0;
   43                for (int i = 0; i < n; i++) {
-> 44                    t += input[i] - s;
   45                }
   46                s += t/n;
   47            }
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e796 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e796 sourceCpp_57725.so`Rcpp::sugar::Mear
   40            s /= n;
   41            if (R_FINITE((double)s)) {
   42                long double t = 0.0;
-> 43                for (int i = 0; i < n; i++) {
   44                    t += input[i] - s;
   45                }
   46                s += t/n;
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e790 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e790 sourceCpp_57725.so`Rcpp::sugar::Mear
   41            if (R_FINITE((double)s)) {
   42                long double t = 0.0;
   43                for (int i = 0; i < n; i++) {
-> 44                    t += input[i] - s;
   45                }
   46                s += t/n;
   47            }
```

```
(lldb) n
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e796 sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e796 sourceCpp_57725.so`Rcpp::sugar::Mear
   40           s /= n;
   41           if (R_FINITE((double)s)) {
   42               long double t = 0.0;
-> 43               for (int i = 0; i < n; i++) {
   44                   t += input[i] - s;
   45               }
   46               s += t/n;
```

If you're looking closely, you'll notice we're now jumping back and forth within this `for` loop. We could keep mashing `n` until we finally get out, but that's a pain (and could be a huge one if we were iterating over a large vector here!). What we want to do is set a new breakpoint a little bit ahead, and then continue execution to that point. Let's do just that, with `breakpoint set --line 46`, and `c` to continue:

```
(lldb) breakpoint set --line 46
Breakpoint 2: where = sourceCpp_57725.so`Rcpp::sugar::Mean<14, true,
(lldb) c
Process 4169 resuming
Process 4169 stopped
* thread #1: tid = 0x70d5, 0x000000010cf7e79f sourceCpp_57725.so`Rcpp
    frame #0: 0x000000010cf7e79f sourceCpp_57725.so`Rcpp::sugar::Mear
   43               for (int i = 0; i < n; i++) {
   44                   t += input[i] - s;
   45               }
-> 46               s += t/n;
   47           }
   48           return (double)s ;
   49       }
```

Cool!

Aside

If you were paying attention as the code snippets passed us by, you'll notice that the Rcpp mean implementation is written to match R's mean implementation, which means it:

1. Stores the intermediate results of the computation in a `long double`, for extra precision,
2. Does two passes over the data, to correct for floating point errors in the first pass.

For those of you who haven't been exposed to the wonders of floating point, or the divide between how computers actually internally represent numbers and how they're actually defined and used mathematically, it can be somewhat surprising – similar to the initial surprise everyone sees with

```
0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

It's nice to understand that, when you write `0.1` into a script, R actually translates that into the closest numeric representation it can. You can get a better peek at these numbers by setting the `digits` parameter:

```
options(digits = 22)
```

```
0.1
```

```
[1] 0.10000000000000000055511
```

```
0.2
```

```
[1] 0.20000000000000000111022
```

```
c(0.1 + 0.2, 0.3)
```

```
[1] 0.30000000000000000444089 0.29999999999999999888978
```

It also means you should be somewhat suspicious of people who claim that they've figured out a way to compute the mean of an array faster than R. Are you computing the *same* mean, with the *same* (or greater) precision, with the *same* handling of missing, or non-finite values? It's unlikely that one can do much better at computing this double-pass mean than R already does, as the compiler would already be generating near-optimal code for this particular form of computing the mean. Of course, I would enjoy being proven wrong...

Wrapping Up

With this, you now have the basic set of survival skills to debug C / C++ segfaults. If you want to learn more:

1. Type `help` within the `lldb` REPL, I find it to be surprisingly useful, and

2. Refer to this cheat sheet for a quick reference to the set of available `lldb` commands (and their `gdb` counterparts).

And, if you keep any set of commands in your head, keep these ones:

- `Ctrl + C` to go back to `lldb` , `c` to go to R,
- `bt` to view the **b**ack**t**race, to see what the current state of execution is like,
- `b <function>` to set a **b**reakpoint,
- `n` to move to the **n**ext line,
- `s` to **s**tep into the next function call,
- `fr v` to list all **v**ariables in the current **fr**ame.

Using your Knowledge for Good

Want to put these skills to use? The next time you're working with a program, or R package, that segfaults when you try to do something, you should:

1. Create a reproducible example (self-contained block of code that reproduces the segfault),

2. Try reproducing within `valgrind` or `lldb` , and

3. If successful, forward the example alongside the associated stack trace and any other information you can discover to the package author / maintainer.

Nothing makes a maintainer happier than a reproducible example with an informative stack trace! (And, conversely, nothing makes a maintainer more upset than seeing "your program crashes sometimes, when I do things, but I'm not really sure when, but it crashed, please fix").

Parting Notes

One thing that drives me insane is that `lldb` does not, in its current release, provide any simple way to run it non-interactively from the command line – you always need to start it interactively, call `run`, and then manually run your script. In other words, `R -d lldb -f script.R` does not work!

Fortunately, this will change in the upcoming release of `lldb`, with a new `--batch` argument. If you are incredibly brave, you can try building `lldb` from source. I have a script here that attempts to automate the process, but I very strongly suggest you read before executing it willy-nilly, as you need to do some code signing to ensure everything works.

After installing that, you'll be able to run `lldb` semi-non-interactively with this little script, which lets you non-interactively run an R script with `lldb`:

```
#!/usr/bin/env sh

## Run an R command within LLDB, in batch mode.
: ${R_HOME="$(cd `R RHOME`; pwd -P)"}
: ${R_EXEC="${R_HOME}/bin/exec/R"}

R_HOME=${R_HOME} lldb --batch --file "${R_EXEC}" -o "process launch -
```

With this, I can execute `r-lldb -f test.R` to start `lldb`, jump into R, run `test.R`, and have it immediately break if a segfault is encountered. Huzzah!

Share and Follow

Tweet

Share

Comments

0 Comments Musings on R and C++ D

Recommend Tweet Share

Start the discussion…

LOG IN WITH

OR SIGN UP WITH D

Name

Be the first

Subscribe Add Disqus to your siteAdd DisqusAdd

DISQUS

3 Question COVID-19 Poll

* **1. In the last week, have you begun any improvement activity?**

- Electronic Equipment
- Gardening
- Household Appliances
- Painting
- Remodeling
- None of the Above

Related Posts

The RStudio macOS Rendering Bug 09 Mar 2019

String Encoding and R 21 Feb 2018

Parent Frames and Backwards Compatibility 04 Jan 2017